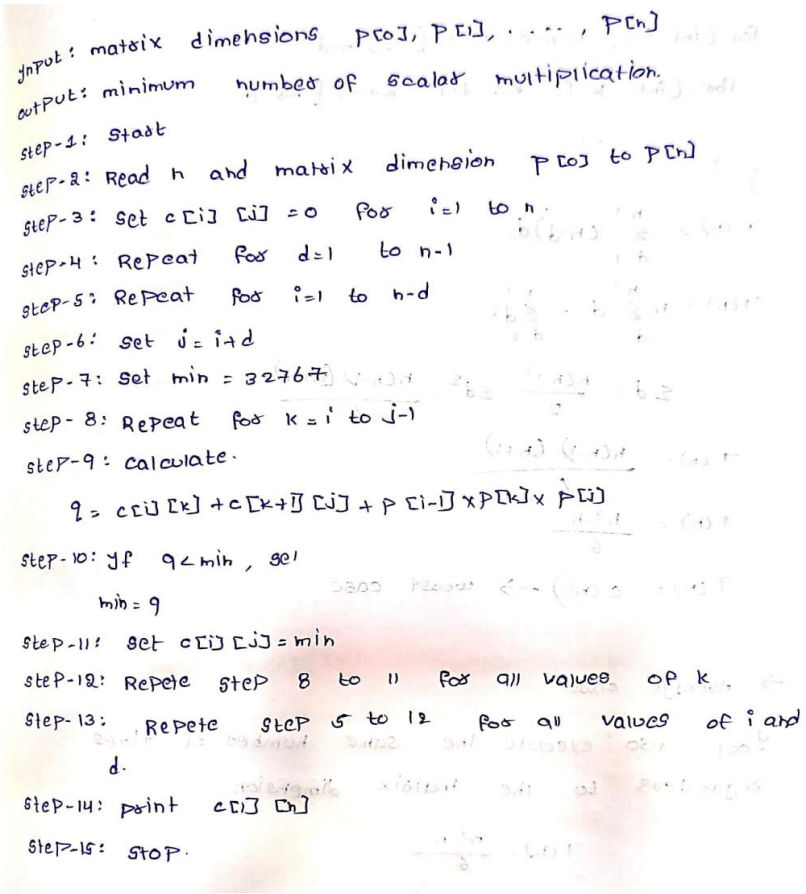
Practical- 6

Aim: To Implementation of chain matrix multiplication using dynamic programming.

chain matrix multiplication

Algorithm:



Program:

```
#include <iostream>
using namespace std;
class MatrixChain
{
private:
    int p[20];
    int c[20][20];
    int n;
public:
    void getInput()
```

```
{
    cout << "Enter number of matrices: ";
    cin >> n;
    cout << "Enter dimensions: ";
    for (int i = 0; i <= n; i++)
    {
        cin >> p[i];
    }
}
void matrixChain()
{
    for (int i = 1; i <= n; i++)
    {
        c[i][i] = 0;
    }
    for (int d = 1; d < n - 1 + 1; d++)
    {
        for (int i = 1; i <= n - d; i++)
        {
            int j = i + d;
            int min = 32767;

            for (int k = i; k <= j - 1; k++)
            {
                int q = c[i][k] + c[k + 1][j]
                    + p[i - 1] * p[k] * p[j];

                if (q < min)
                {
                    min = q;
                }
            }
```

```
            c[i][j] = min;
        }
    }

    cout << "\nMinimum number of scalar multiplications = "
         << c[1][n] << endl;
}

void display()
{
    cout << "\nDP Table:\n";

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (j < i)
                cout << "0 ";
            else
                cout << c[i][j] << " ";
        }
        cout << endl;
    }
}
};
int main()
{
    MatrixChain obj;
    obj.getInput();
    obj.matrixChain();
    obj.display();
    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-6\chain matrix.exe"
Enter number of matrices: 4
Enter dimensions: 3
2
4
2
5

Minimum number of scalar multiplications = 58

DP Table:
0 24 28 58
0 0 16 36
0 0 0 40
0 0 0 0
```

Time complexity:

```
for (int d=1; d<n; d++) {  -> n-1
for (int i=1; i<=n-d; i++) {  -> n-d
  for (int k=i; k<=j-1; k++) {  -> d
} }
```

$$T(n) = \sum_{d=1}^{n-1} (n-d)d$$

$$T(n) = n\sum_{d=1}^{n-1} d - \sum_{d=2}^{n-1} d^2$$

$$\sum d = \frac{n(n-1)}{2} \qquad \sum d^2 = \frac{n(n-1)(2n-1)}{6}$$

$$T(n) = \frac{n(n-1)(n+1)}{6}$$

$$T(n) = \frac{n^3-n}{6}$$

$$T(n) = O(n^3) \rightarrow \text{worst case}$$

→ Average case

loop also execute the same number of time regardless to the matrix dimension.

$$T(n) = \frac{n^3-n}{6}$$

$$T(n) = O(n^3)$$

Name:M.Ravi Kiran Reddy
Enrollment Number:92460118230

Conclusion: The Chain Matrix Multiplication problem was successfully implemented using Dynamic Programming. The algorithm finds the optimal order of matrix multiplication and minimizes the number of scalar multiplications. It stores the solutions of smaller subproblems in a DP table and uses them to solve the complete problem efficiently.

Name:M.Ravi Kiran Reddy
Enrollment Number:92460118230